

Master Script

What, when, how, and help — for the entire day, in one document

This is the one to hold. Every segment below follows the same shape: **What** it is, **How** to run it, what to **Say**, where the **Help** (hints/prompts) lives, and — where relevant — the actual **Answers** and what to do **If Stuck**. You shouldn't need a second document today.

Jump to:

Hour 1 — Bug Hunt (0:00–1:00)

Level Up (1:00–1:30)

Build: Tutorial, Plan & Create (1:30–3:35)

Test & Show (3:35–4:00)

0:00 Welcome

10 min

WHAT

Intros, get everyone into their Codespace, ice breaker.

HOW

- Share the repo URL on screen
- Pair people up — mix ages where you can

SAY

"Today you're going to build a real feature for a banking app — from idea to working demo. We'll start by doing what every developer does first: fix what's broken."

0:10 Copilot demo

15 min

WHAT

Live demo of inline completions and Copilot Chat.

HOW

- Type a comment, wait for ghost text, press Tab
- Open Chat, ask it to explain a snippet of code

SAY

"Copilot is a very fast autocomplete that's read most of the internet. It's not always right. Your job is to understand the problem — Copilot helps you write it faster."

0:25 Bug tutorial — everyone together

10 min

WHAT

Everyone follows `how-to-find-fix-bugs.html` — a worked, made-up example bug, not one of the real 10.

HOW

- Walk through the fake `sayHi()` / `sayHu()` example together
- Land the 4-step method: Look → Use → Read → Fix → Test

IF STUCK

No spoilers here — this example is deliberately not a real dashboard bug, so it's safe to fully solve on screen.

0:35 Independent bug hunt

20 min

WHAT

Same method, applied to the real `bank-dashboard.html` — 10 planted bugs.

HOW

- Project `bug-hunt-screen.html`, step back
- Remind them to log finds in `bug-test-plan.html`

HELP — HAND OUT IF STUCK

"Are there any bugs in this code?"

"I clicked this button and nothing happened — what could cause that?"

"Why might a number show up as NaN instead of a value?"

ANSWERS — ALL 10 BUGS

#	BUG	FIX
1	Missing closing tag	Add <code></div></code> after balance footer
2	Invisible balance	<code>color: #0F1340</code> → <code>#FFFFFF</code>
3	Wrong transaction sign	Coffee: <code>credit</code> → <code>debit</code>
4	Wrong input type	<code>type="text"</code> → <code>type="number"</code>
5	Button typo	<code>onlick</code> → <code>onclick</code>
6	Wrong operator	<code>balance + amount</code> → <code>- amount</code>
7	Element ID typo	<code>"balance-display"</code> → <code>"balance-dispay"</code>
8	NaN on empty input	Add <code>isNaN()</code> check before calculating
9	Wrong interest formula	<code>* rate</code> → <code>* (rate/100)</code>
10	No overdraft check	Add <code>if (amount > balance)</code> guard

If a group asks Copilot to "fix everything at once": it'll likely miss 8, 9, 10 (missing logic, not broken code) and 3 (a data error, not a syntax one). Use it as a live teaching moment if it happens — stop the room, look at what it missed together.

If Nova's restraint is genuinely too much for one student: only affects that one person since each has their own repo. Rename their `.github/copilot-instructions.md` on `github.com` → rebuild their Codespace container → Copilot reverts to normal for them only.

0:55 Share out

5 min

HOW

Ask 2–3 people: "What did you find? How did Copilot help?"

SAY

"That's the debugging loop — notice, investigate, fix, test. Copilot speeds up the middle part, but you're still the one running the loop."

1:00 The reveal + concepts

8 min

WHAT

Fast facts about GitHub, Codespaces, and Copilot's scale — then land what a Codespace actually is.

SAY

"You just used GitHub Copilot to debug a real banking app. It was trained on over a billion lines of code. And it still gets things wrong sometimes. That matters."

1:08 Round 1 — Fix the Bug

7 min

WHAT

`challenge.js` already has a buggy interest-rate function pre-loaded. Ask Copilot Chat to fix it, then prove it with Node.

HOW

- Open `challenge.js` — it's already there, no typing needed
- Ask Copilot Chat to review and fix it
- Add `console.log(calculateMonthlyPayment(1000, 5, 1));`, save
- Terminal: `node challenge.js` — see the real number

HELP

"Can you review this function for bugs? Pay attention to how the interest rate is being used."

ANSWER

`monthlyRate = annualRate / 12` → should be `annualRate / 100 / 12`. 5% treated as 5, not 0.05.

If Copilot refuses / flags "SQL injection": shouldn't happen here since the code's already in a real file — but if it does, confirm the function is actually saved in `challenge.js` before asking Chat about it.

1:15 Round 1 recap

3 min

SAY

"Copilot wrote that bug. Then Copilot found and fixed it. That's not a contradiction — write fast, review carefully, use it in both directions."

1:18 Round 2 — Comment Race

3 min

WHAT

Same `challenge.js`. Type a comment, Tab-complete, prove it with Node.

HOW

- Comment: `// function that converts an amount in pounds to euros, rate 1.17, rounded to 2 decimal places`
- Tab to accept, add a `console.log`, run `node challenge.js`

1:21 **Round 3 — Best Prompt**

3 min

WHAT

One Copilot Chat message only. Best result wins, proven via Node.

SAY

"The difference between a great result and a mediocre one came down to how specific the prompt was. This is prompt engineering."

1:24 **Golden rules + bridge**

6 min

SAY

"Three rules before we build: be specific, always test it, ask Nova to explain anything you don't understand. Now — let's build something real."

1:30 Feature tutorial — everyone together

20 min

WHAT

Everyone follows `how-to-build-your-feature.html` — builds a Savings Tracker from a genuinely blank file. 10 steps: skeleton, colours, header, form, progress bar, logic, wiring.

SAY

"One: write a comment describing the next small thing. Two: let Copilot suggest, press Tab. Three: save, check Live Server. Four: if it's wrong, ask Nova why."

IF STUCK

Y11/Y12–13 have exact comments hidden behind a reveal button on this page — encourage them to try writing their own first.

1:50 Brainstorm

15 min

WHAT

Popcorn round, then pick an idea — different from the tracker they just built.

IF STUCK

Point at the back-pocket gallery on `brainstorm-screen.html` slide 2 — just not the Savings Tracker.

2:05 Plan on paper

10 min

SAY

"Keyboards down. Every minute you plan saves five minutes of confused coding."

2:15 Feature creation — first half

25 min

WHAT

Everyone creates a new file, copies colours/header from `practice-feature.html` (built in the tutorial).

TARGET BY 2:40

Heading, inputs, button, opens in Live Server without errors. Ugly is fine.

2:40 Checkpoint

5 min

HOW

Every group: one sentence, what's working / what's next. Fill in Progress Tracker.

HELP BY YEAR GROUP

Y9–10:

"Why isn't my progress bar changing width?"

Y11:

"Review this function for edge cases I might have missed"

Y12–13:

"How would you refactor this to be more maintainable?"

At 3:20, if any group isn't working yet: "Strip it back. One input, one button, one number on screen." Say it to the whole room.

3:35 Feature testing

10 min

SAY

"Code freeze. Open your Feature Test Plan and try to break your own feature on purpose — empty fields, negative numbers, huge numbers."

3:45 Show & tell

15 min

HOW

2 min each. Room answers "what did you like?" after each.

If a demo breaks live: "Congratulations — a live demo failure. That's the most authentic developer experience of the day." Show it in the editor instead.

YOUR CLOSING

"Two hours ago you built one feature together, step by step. Then you described your own idea, Copilot helped you build it, you tested it, you fixed it, you shipped a demo. That loop — describe, build, test, fix — is the actual job. The tools will keep changing. The loop won't."

If something isn't covered here, it's on the Troubleshooting Card or the "If It All Falls Apart" page. If it's still not covered — you already know more about this than you think you do.